YAMIR

A look at running a MANET on Linux workstations and Android mobile devices

Copyright © 2012

COF

All Rights Reserved

Abstract

Mobile phones these days have moved well beyond simple devices that just make voice calls with many models now having the computing power and wireless connectivity to rival netbook computers earning them the title of smart-phones.

And yet for all their wireless connectivity they still depend for the most part on a fixed or centralized network infrastructure for voice and data communications. To make a call or send data over the internet a user will invariably have to use some form of fixed or centralized infrastructure in the form of a Wi-Fi access point or a cell tower belonging to a mobile phone operator.

For reasons including economic underdevelopment, geographic remoteness, or impact from recent disasters, the fixed networking infrastructure may be severely limited, expensive to use, or simply nonexistent. In such situations having access to an alternative infrastructure that can be quickly assembled, is self-maintaining, and is free to use may be the only viable means of communication. So why not use the phones themselves ?

This project will look at how the wireless capabilities of devices such as smart-phones can be used to create ad-hoc or decentralized networks where the devices themselves form the physical network infrastructure. Such networks are known as mobile ad hoc networks (MANETs) and this project will examine the issues behind MANETs, the technologies and algorithms that can used as well as the work involved in providing a solution by actually building a MANET out of existing smartphones.

Table of Contents

YAMIR....1
Copyright © 2012....2
Abstract....3
Chapter 1 Introduction....5
1.1 Motivation....5
1.2 Objectives....6
1.3 Project Outline....6
Chapter 2 Background and related work....7
2.1 Wireless Access....7
2.2 Addressing....8
2.3 Routing....10
2.4 Service Discovery....11
2.5 Voice over IP....12
2.6 Related Work....13
Chapter 3 Design Overview....14
3.1 Platform....14
3.2 Wireless Access....14
3.3 Routing....14
3.4 The DYMO Routing Protocol....15
3.5 Netfilter - Linux kernel network stack monitor....16
3.6 YAMIR – MANET IP router....16
3.7 Administration....17
3.8 SIP Client....17
3.9 Architecture Diagram....18
Chapter 4 Implementation....19
4.1 Prerequisites....19
4.2 Building....20
4.3 kyamir - Linux kernel module....20
4.4 yamird – userspace daemon....22
4.5 Android application....24
4.6 SIP Client....26
4.7 Issues....27
Chapter 5 Conclusions and future work....29
Bibliography....30

Chapter 1

Introduction

This project presents an alternative to using a fixed infrastructure for communication using a Mobile Ad Hoc Network (MANET). A MANET is a self organizing network composed of nodes that are both free to move around as well as join or leave the network at will. This project aims to demonstrate that by using a MANET architecture a free and independent telecommunications network can be built out of consumer devices such as smart-phones and laptops. This chapter will examine the motivation behind the project, its objectives and provide an overview of the rest of the paper.

1.1 Motivation

The most visible form of modern day communications these days is the mobile phone which is increasingly becoming more of a mobile data link to the internet rather than something just to make voice calls with. And yet even though the mobile phone is a wireless device it still exhibits one characteristic in common with the land-line based telephone service (POTS) its intended to replace, namely that it requires access to a large centralized infrastructure that is neither cheap to build or free to use.

In urban areas where there are sufficient numbers of people willing to pay for a mobile phone service the cost of building and maintaining such an infrastructure can be justified but for developing societies or remote sparsely populated regions the technical difficulties or the economic return may be too poor, resulting in a limited or simply non existent network infrastructure. Even in areas where fixed network coverage is normally adequate it can suffer a degraded or complete loss of service due to a recent disaster or even by large concentrations of people attending social gatherings such as sporting or music events.

A MANET allows a data network to be quickly setup in areas where a fixed infrastructure is unavailable or the cost is prohibitive. As they are self organizing they do not require the same level of maintenance required as fixed infrastructure. They can be easily extended and the failure of one or more nodes does not necessarily mean the failure of the entire network.

Emergency services can make use of MANET-powered walk-talkies [1] where existing infrastructure has been destroyed or is unavailable due to power loss. MANETs also have many critical military applications, ranging from drones to mobile command-and-control units, where the rapid deployment of a resilient and authenticated mobile communications network is vital to operational successes.

Such organization can afford to have specialist equipment built to order that can run MANETs but the widespread availability of consumer devices with wireless connectivity now makes it possible for anyone to build MANETs as well. For example a local community in developing society could setup their own local phone network and share data using older laptops and phones using a MANET[2].

1.2 Objectives

This project hopes to achieve the following:

- Explain the technologies behind MANET
- Discuss the problem experienced by MANETs and various solutions
- Examine related work
- Design and implement a functioning MANET
- Conclusions and further work

1.3 Project Outline

The remainder of the report is as follows.

Chapter 2 explores the problems involved in building and running a MANET, possible solutions and other related work.

Chapter 3 proposes a design for building a MANET outlining the required components, algorithms and architecture.

Chapter 4 describes in depth how the design was implemented as well as any issues encountered.

Chapter 5 examines the results of the implementation, makes some concluding remarks and discusses any future work.

Chapter 2

Background and related work

MANETs as described earlier, can be formally described as a communication network using wireless links where nodes are mobile and not tied to any fixed infrastructure.

Nodes are not just the source or destination of data traffic; they also act as intermediate routers, relaying traffic on behalf of nodes that are not within direct wireless range of one another.

This chapter examines four primary challenges affecting MANETs: wireless access, routing, address configuration and service discovery. Additionally it explores how voice calls can be made over a MANET, reviews related ad-hoc technologies, and finally takes a look at several existing MANET implementations.

2.1 Wireless Access

For nodes to communicate with one another, they need a wireless protocol. The two most common found in laptops and smartphones are Bluetooth and Wi-Fi.

Bluetooth was developed as a cable replacement technology for connecting electronic devices such as headsets and keyboards over a short range low power radio interface.

The basic unit of a Bluetooth setup is a Piconet which is made up of a master node and up to seven slave nodes with a theoretical maximum range of up to 100m depending on the class of master.

More than one Piconet can exist in the same area and they can be connected together via a bridge node forming an interconnected Piconet known as a Scatternet. Piconets are a form of ad-hoc network as they can operate without any fixed infrastructure so long as one device is willing to be the Master.

Although a master can communicate directly with any slave, slaves can only communicate with the master and not directly with each other even if they are within wireless range. Although class 1 Bluetooth allows a theoretical wireless range of 100m most consumer devices such as mobile phones only support class 2 which limits their maximum range to 10m. The maximum data throughput of Bluetooth radio is around 2.1 Mbps although the high-speed update introduced in version 3 of the protocol can boost this up to 24 Mbps by using 802.11 WLAN radio to actually carry the data.

Wi-Fi, officially called IEEE 802.11, was originally developed to allow computers, such as laptops, to connect to existing LANs without the need for cables.

In 802.11 the basic unit is called a Basic Service Set (BSS), which represents a collection of network elements known as stations (STA), and access points (AP).

Two types of network modes are supported namely infrastructure mode and independent mode also known as IBSS. With infrastructure mode the stations within a BSS all associate with an access point (AP) which they then use to communicate with each other or another network.

In independent (ad-hoc) mode stations do not use an access point and can communicate directly with each other so long as they are within wireless range. Theoretical wireless range for stock antennas varies between 70 to 250m depending on transmission method (a/b/g/n). Although 802.11n standard supports transmission rates up to 300Mbps the IEEE standard specifies that ad-hoc mode only needs to support 11Mbps.

Wi-Fi Direct [3] is a recently introduced extension by the Wi-Fi alliance to allow peer-to-peer connections between consumer devices.

Wi-Fi Direct introduces a group owner and client devices to a 802.11 network. A group owner allows one-to-many links in a manner similar to a Bluetooth master enabling it to setup an ad-hoc network with slave devices.

In addition group owners can talk to each other directly forming a true peer to peer network. For older legacy devices the group owner can act as a soft access point allowing these devices to associate with the group owner and form an infrastructure BSS.

2.2 Addressing

Nodes must be assigned an address when they join a network. In IP-based routing, this address needs to be unique to ensure data packets reach their intended destinations.

When the number of nodes is small, static IP address assignment can be used in both private or public networks[4]. Fixed addressing greatly simplifies address configuration and service location, as nodes do not depend on a discovery mechanism to begin communication. For certain services, fixed address are essential; for instance, the Domain Name System (DNS) requires nodes to know the IP address of a name server to perform lookups.

The more common approach is dynamic or auto-address configuration. In wired or infrastructure-based wireless networks, a fixed server is assigned the task of distributing unique address from a pool. However, in a MANET - where nodes randomly join and leave - a dynamic address configuration protocol must be in place. This ensures that addresses are unique and valid for the duration of a node’s presence and accounts for challenges like network partitioning and merging.

Dynamic address configuration protocols can be divided up into two categories: Stateful and Stateless configuration [5].

Stateful address configuration is where nodes keep track of the state of the network by maintaining a table of available addresses. Because they keep track of the network address state they are considered conflict-free.

Stateful protocols involve client nodes broadcasting address requests which are then responded to by other server nodes. In fixed networks the Dynamic Host Configuration Protocol (DHCP) is the most common example. However using one central server for address assignment is not appropriate for a MANET as if the node becomes unreachable the entire network can fail.

For MANETs one often quoted example is MANETConf which uses distributed allocation scheme [6]. In ManetConf each network nodes keeps a list of addresses in use. When a new client node requests an address via a broadcast it will choose the first server that responds and then request from it a unique address. Before the server responds it will send a network broadcast to check if the chosen address is available for use and then wait for acknowledgment from all known nodes. Network partitions are detected if no acks come from addresses marked as in use. If the server receives a positive ack from all other nodes it will assign the address to the client node and then broadcast this fact throughout the network so that the rest of the nodes keep their tables updated. Stateful schemes such as MANETConf have the disadvantage where the flooding of control traffic (address config) to network can reduce the available bandwidth for actual data traffic.

In stateless address configuration nodes manage their own ip address instead of maintaining an address allocation table. They initially select their own address either randomly from a known range or use an algorithm involving some unique attribute such as their MAC address. They then use a duplicate address detection (DAD) process to check if the chosen address is actually unique.

An example of a stateless address protocol in fixed infrastructure is the Stateless Address Auto-Configuration (SLAAC) used in IPV6. In SLAAC a client uses its MAC address to create an ipv6 address which it then broadcast over the network interface. This requires that all nodes are within a single hop and the generated address is unique. If a duplicate address is detected then address configuration fails and it has to be manually configured. This is unsuitable for MANETs which are multi-hop based and address conflict is a distinct possibility.

Instead nodes can employ one of three schemes to ensure conflict free address allocation. In strong or query based DAD a node will check if a selected address is use by using a broadcast or sending an ICMP message to the selected address. If no response is received the address is considered free to use.

In weak DAD a nodes can tolerate duplicate addresses for a time by generating an additional key to use in conjunction with the chosen IP address. If the routing protocol detects that two keys in a routing message have the same IP address then it can prevent packets being sent to the duplicate address and optionally inform nodes of the conflict. In Passive DAD nodes simply monitor passing routing requests for hints about what IP addresses are in use.

2.3 Routing

In an ad-hoc network where nodes are within range of one another, they can communicate directly using single-hop routing by just transmitting packets over the wireless medium.

When nodes are not within range of each other, they must use their neighbours (if they are willing) to forward the packets on their behalf towards their intended destination. In such multi-hop environments, a routing protocol is required to establish how nodes forward IP traffic from source to destination.

Distance vector and link state protocols - such as RIP and OSPF - are designed for designed for use static wired environments where resources such as bandwidth,memory and power are relatively cheap compared to nodes in a wireless ad-hoc environment.

The highly dynamic nature of a MANETs topology, where nodes randomly appear and disappear, combined with the fluctuating quality of wireless links, makes standard link-state state and distance-vector wired routing protocols unsuitable for use in such an environments[7].

The IETF MANET working group has created routing protocols that can be divided into two main categories: proactive and reactive routing protocols [8].

Proactive, or table driven, protocols require nodes to maintain routing information for the entire network by periodically exchanging updates, typically by flooding the network with routing control packets.

They are essentially derived forms of the distance-vector or link-state algorithms used in wired networks but are optimized for MANET environments. Their primary advantage is the elimination of route discovery delay, as routes to all destinations are pre-established and maintained. However, their main disadvantage is that in highly unstable networks, they risk saturating the network with routing control traffic at the of expense of actual user data.

One well known example is Optimized Link State Routing (OLSR) [9]. It is a modified version of OSPF (Open Shortest Path First) where instead of flooding the network with route updates a node selects a subset of its one-hop neighbours known as multi-point relays (MPR's) where link quality is judged best. It is these MPR's that perform routing and routing updates on behalf of the entire network.

Reactive, or on-demand, routing protocols do not maintain a constant map of the network topology. Instead, they only discover and maintain routes between nodes as needed. Their main advantage is that they require fewer resources (such as memory and CPU) than proactive protocols and perform better when the network is unstable. However, their main disadvantage is that applications may experience high initial latency (long delays) while route discovery takes place.

Three well-known example are Dynamic Source Routing (DSR), AdHoc On-Demand Distance Vector (AODV) and Dynamic MANET On-demand (DMYO) [10]. Although there are implementation differences all three utilize the same concepts of route discovery and route maintenance.

In route discovery mode, an initiator broadcasts a Route Request (RREQ) message to its neighbours, who in turn broadcast it to theirs until it reaches the target node. The target then sends a Route Reply (RREP) back to the initiator, typically along the same path the request travelled. In route maintenance, if a node receives a data packet for an unknown or broken route, it sends a Route Error (RERR) message. All nodes receiving the error message will then mark any routes that use that address as unusable.

2.4 Service Discovery

In a MANET using dynamic or auto-address configuration, nodes must be able to discover the addresses of others to establish contact or utilize specific services [11].

For example when making a SIP or Voice over IP (VoIP) call in a network with dynamically assigned address, the called party must register its current address with a server that accessible to the calle. This allows the called to determine where to direct the VoIP traffic.

To locate a server, a node must use a Service Discovery Protocol (SDP).
Examples in the wired world include Jini and UDDI, while UPnP is common for wireless devices. These protocols involve the periodic flooding of the network with advertisements from service providers nodes, discovery requests from clients nodes or a combination of both. Nodes may also choose to listen to the SDP traffic to capture relevant data.

SDP can be divided into directory-based or directory-less architectures.
In a directory-based architecture, specific nodes maintain a list (or registry) of all available services in the network. This registry can be centralized on a single node or distributed across several. Conversely a directory-less architecture requires all nodes to broadcast service discovery requests to locate a resource as needed.

To reduce the number of broadcasts, some directory-based SDPs designed for MANETs attempt to piggyback service information onto the routing protocol messages or extend the routing protocol itself. In the SipHoc protocol [12] all SIP address registrations from client applications are accepted by a proxy running on the local device. As routing messages are transmitted, the protocol attaches extra service information to inform other SipHoc nodes of the address.

An example of a directory-less SDP is emergency response protocol [13] which utilizes location awareness and client broadcasts to locate and reserve necessary services.
The MANET is divided into geographical zones, where resources - such as emergency personal and equipment – are connected via PDAs. Resource are assigned service categories (e.g surgeon, nurse,driver, or supplies) and service requests are classified by urgency such as severe, medium, or low. When a client issues a request, its is broadcast through the network, and matching provider respond with their location and availability. If a service is not found in requesters current zone, nearby zones are queried. The protocol can then issue a reservation request for the closest match, informing personnel or reserving equipment for use.

2.5 Voice over IP

To facilitate a call over a MANET, a Voice over IP (VoIP) service must be running on both the source and destination nodes.

One widely adopted protocol is the Session Initiation Protocol (SIP) [14] .

SIP was developed by the IETF as a simpler alternative to ITU's H.323 protocol.
It is modelled after HTTP and uses the same text-based MIME encoding used in email messages. The protocol can use either UDP or TCP as the transport layer.

In SIP, callers and calles are known as User Agents (UA).
A UA can be a physical device, such as a VoIP enabled desk phone or smartphone, or a software application (aka softphone) running on a laptop or desktop computer.

In the simplest case where the calling party knows the IP address of the called party, a UA initiates a call by sending an INVITE message directly to the calle's UA. The called party responds with a 180 Ringing message followed by a 200 OK (when the user answers), which the caller then acknowledges with an ACK message. A media session is then established between both parties to exchange voice traffic via the Real-time Transport Protocol (RTP). Either party can terminate the session with a BYE message which the other acknowledges.

In the case where the UA does not know the address of the called party it must contact a SIP proxy or redirect server to perform the call setup or return the called party's IP address. In either case the called party must have registered its own ip address with another proxy that can be contacted by the callers server.

2.6 Related Work

We introduce here wireless mesh networks which are closely related to MANET''s and the Serval project which provides mesh based phone networks.

Wireless Mesh Network

Wireless Mesh Networks (WMN) are another type of wireless ad-hoc network similar to MANETs but without the problems of constantly moving nodes. The IEEE have introduced a draft standard 802.11s [15] that adds mesh support to existing 802.11 networks by adding multi-hop frame forwarding and path-selection capabilities at the data link or MAC layer. This solves many of the problems experience by existing WMN or ad-hoc implementations that do multi-hop packet forwarding and routing at network or IP layer. IP layer ad-hoc protocols have a indirect view of the radio environment based on IP packet monitoring whereas a MAC layer implementation can maintain very accurate link metrics as well as maintaining quality of service and congestion controls for the radio interface.

802.11s introduces the concept of a Mesh Basic Service Set (MBSS) which consists of three network elements, a Mesh Point or Station (MP), a Mesh Access Point (MAP) and a Mesh Point Portal (MPP). A Mesh Point implements a MAC layer multi-hop forwarding and path discovery. A Mesh Access Point is a normal 802.11 AP with Mesh capability and a MPP acts as a direct bridge to other non 802.11 networks. The path selection algorithm employed is called Hybrid Wireless Mesh Protocol and supports both a proactive tree based method as well as reactive (on-demand) method based on the MANET AODV protocol.

Although it began as draft protocol 802.11s has already been used by the OLPC foundation as the underlying technology for mesh networks formed by their XO laptops which are for use in developing countries.

The Serval Project

The Serval Project is a mesh based phone network developed by a team from the Australian Flinders University [16][17]. The project aims to provide a free infrastructure-independent phone system that can operate in places where people lack access to a telecoms network such as remote regions or disaster areas.

The project uses android based mobile phones running a sip client (sipdroid) and can make use two IP based proactive routing protocols either the manet OLSR or the Better Approach To Mobile Ad Hoc Routing (BATMAN). The BATMAN protocol was developed by the Freifunk community based in Germany in response to perceived shortcoming to OLSR protocol ranging from its complexity to its actual performance in real world scenarios outside of simulators [18][19] .

The protocol can be described as based on the idea of stigmergy exhibited in ants and termites where traces left behind by actions of one agent change the behaviour of others to the benefit of the whole group. The protocol involved nodes broadcasting hello or originator messages (OGM) to their neighbours. The OGM consisting of an originator address, sequence number and node sender address which a neighbour will change to its own before rebroadcasting. Upon receipt of its own message (identified by originator address) a node can judge by its reception peed and frequency which neighbouring links are the best hop for sending data. A more recent version of the BATMAN protocol called batman-adv (not used by Serval) was developed which is now located in the data link layer for the same quality of server reasons behind 802.11s [20].

Chapter 3

Design Overview

Building a MANET from scratch requires a large number of elements incorporating many features discussed in the previous chapter such as Wireless Access, Addressing, Routing and Service Discovery, including some administrative tools and applications that make use of it. As such it was decided that only a subset of these features could be realistically built in given the time available and so fixed addressing was used for the MANET removing the need for address auto-configuration and service discovery. This then leads to a system design involving Wireless Access, Routing, Administrative tools, a VOIP application as well as a choice of what platform to run it on.

3.1 Platform

We choose Linux as our target platform as some initial research suggested some fairly low-level network stack operations would be needed requiring access to the kernel source. This meant that code could be fully tested on some laptops before integration onto smartphones. Choosing Linux meant Android was the default smartphone choice as these are widespread ARM based platforms running the Linux kernel.

3.2 Wireless Access

As outlined in Chapter 2, the choice of wireless technology for laptops and smartphones typically comes down to Bluetooth or IEEE 802.11. Although Bluetooth is available on most mobile handsets is not a standard feature on all laptops. The 802.11s mesh standard was considered outside the project scope, as this project is a study of an IETF MANET which operates at layer 3 (IP-based). Furthermore, since Wi-Fi Direct (as of 2012) is experimental protocol limited to certain high-end handsets, 802.11 IBSS (Ad-hoc mode) was selected as our wireless access method.

3.3 Routing

Routing in MANETs can be divided into 3 core areas: packet forwarding, route discovery and route maintenance.

The Linux kernel handles packet forwarding by consulting its routing tables to determine the egress interface for each IP packet.

A MANET IP router must handle route discovery and route maintenance. To operate correctly, the router must monitor packet activity within the Linux network stack. This necessitates two core components: a kernel-level packet interceptor and a user-space routing daemon.

3.4 The DYMO Routing Protocol

Because our router has to run on resource constrained nodes such as smartphones, we opted for a reactive routing protocol. We chose to implement the DYMO (Dynamic MANET On-Demand, draft 21) protocol because it was the most recent reactive protocol produced by the IETF MANET working group and is simpler to implement than AODV [22]. As mentioned earlier, DYMO can be divided into two operations: route discovery and route maintenance.

Route discovery is triggered when a data packet needs to sent to an IP destination address (daddr) for which there is no entry in the routing table.

A route request (RREQ) message is constructed containing the target address (daddr) and nodes own IP address as the origin. The packet also includes a hop limit and a sequence number, which are used to limit network flooding and detect duplicate or stale requests. This request is encoded using the PacketBB (rfc 5444) format [23] and transmitted as a UDP packet. The UDP packet source IP address is set to the local node, the destination port set to 269, and the destination IP address is set to the MANET link-local multicast address 224.0.0.109.

The router then sets a timer and waits for a reply. All routers that receive the request will after ensuring its not a duplicate, invalid or stale will update their routing tables to indicate that IP traffic can be routed back to the origin using the source IP address of the UDP packet that the request arrived in on. If the receiving node local address does not match the target address it will relay the request using another link-local multicast after first checking the hop limit has not been exceeded (in which cast it drops the request). This is the method by which a data route is built for the original destination address (daddr) that triggered the route discovery.

If the receiving node local address match's the target address it will create a Route Reply (RREP) message with the target address set to the request origin and the origin address to its own local address which should be the same the request target address. It will then send it a MANET encoded UDP packet with the destination IP address set to one of of its available routes, which is usually the source IP address of the UDP packet the request arrived in on. This will then be unicast relayed back by intermediate nodes using an appropriate route (usually the one first established by RREQ) back to the origin node which upon reception will cancel its timer, add a new route to its table and allow the original data packet be forwarded using the new route. In the event of no reply the timer will expire and the router will attempt a number of retries which if all unsuccessful will cause the original IP packet to be dropped and an ICMP destination unreachable error sent to the application that tried to send the packet.

Route maintenance is triggered when routes expire due to non-use or if a router receiving a data packet for which it has no route. All route entries have an accompanying timer which gets reset every time they are used by a data packets. Upon actual expiry the entry will be discarded from the routing table. If a router received a data packet for which there is no route it will discard the packet and send a route error (RERR) message recording the unreachable address to the the MANET multicast address. All routers that receive the multicast will remove any routes that contain the unreachable address as the next hop.

3.5 Netfilter - Linux kernel network stack monitor

The Linux kernel provides an packet filtering framework called netfilter for monitoring its network stack[21]. This framework provides a hook API where user defined functions can be called as IP packets traverse the kernel network stack.

Netfilter defines 5 stages where a used defined function can be called.

1) NF_INET_PRE_ROUTING is where packets have just arrived into the stack before any routing decisions have been made.
2) NF_INET_LOCAL_IN called after routing has decided if a packet is destined for a local process.
3) NF_INET_FORWARD is called if a packet is to be sent out onto another network interface.
4) NF_INET_LOCAL_OUT is called when a local process is sending a packet before any routing decision is made.
5) NF_INET_POST_ROUTING is called after all routing decisions have been made and the packet is being out of the stack.

When a netfilter hook is called it can return one of the following codes

- NF_ACCEPT - kernel keeps packet and continues stack traversal.
- NF_DROP - kernel discards packet, stops stack traversal
- NF_STOLEN – hook has taken control of the packet, stop traversal
- NF_REPEAT – hook requires kernel to call it again

3.6 YAMIR – MANET IP router

YAMIR (Yet Another Manet IP Router) is the core of our project.

This IP router uses a hybrid design where a kernel module detects route requirements and a user-space daemon performs route discovery and maintenance.

- **kyamir** kernel module using netfilter hooks to intercept IP packets
- **yamrid** user-space daemon using DYMO for discovery and rtnetlink for maintenance

We decided on the kernel module approach to network stack monitoring even though there is a user-space variant called libnetfilter_queue.

As libnetfilter_queue would have required all IP network packet to be sent up to user-space we felt this would have negatively impacted the performance of any real time traffic such as SIP. For communication between the kernel modules and user-space code yamir makes use of the Linux Netlink socket interface which allows full-duplex communication between them.

3.7 Administration

Configuring Ad-hoc mode on a Linux 3.3 laptop can nr done using the iwconfig tool and a supported USB WiFi adapter like the Edimax 802.11n (EW-7612UAn)

e.g.
```
 iwconfig wlan0 down
 iwconfig wlan0 mode ad-hoc
 iwconfig wlan0 essid “yamir”
 ifconfig wlan0 192.168.1.1 up
```

Android application

For Android devices like the HTC smartphones it is not quite so straightforward.

The Wi-Fi Manger on android handsets filters out all IBSS (Ad-hoc) networks from the scan results reported by wpa_supplicant making it impossible to configure 801.11 ad-hoc mode using the standard android framework.

Various solutions to the problem have been proposed ranging from modifying the wpa_supplicant tool to report IBSS networks as infrastructure access points to actually forcing wpa_supplicant to associate to an ad-hoc network and then try to prevent WiFI manager from changing it.

As our MANET requires a permanent reliable network connection on each node we opted for an alternative solution which takes charge of the Wi-Fi interface while the MANET is running preventing the android Wi-Fi manager and other apps from interfering with it.

Doing this requires a rooted handset. Rooting involves modifying the phone's firmware to grant applications superuser (root) privileges, similar to administrative access in Linux. This allows applications to run restricted operations that are blocked by the default factory settings.

The process is different for every handset but an internet search or look at http://androidforums.com quickly turns up the appropriate guide.

For android the administration tool is an android app made up of two parts. a backend comprising a set of shell scripts and binaries needed for starting and stopping the MANET and a GUI frontend for both entering network settings and controlling the MANET.

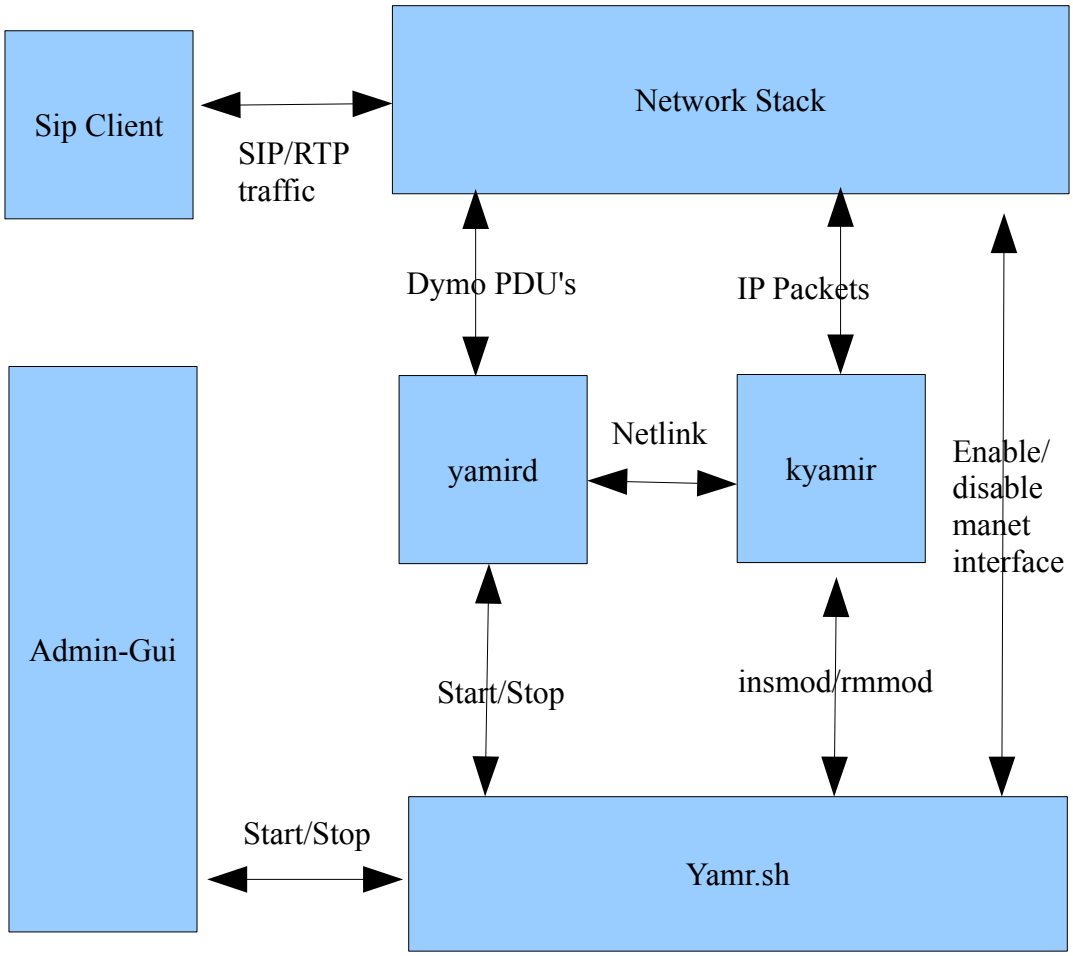
3.8 SIP Client

To make voice or video calls over IP, a SIP client is required.

Three of the most popular clients for Android are SipDroid, cSipSimple and Linphone.

We choose Linphone because it offers version for both Android mobiles devices and Linux workstations. Furthermore, Linphone is open-source, allowing for modifications to its codebase to support the use of our MANET.

3.9 Architecture Diagram



1. SIP Client - Linphone Android app.
2. Admin GUI - yamir Android Application
3. Admin Backend – yamr.sh, yamird and kyamr.ko

Chapter 4

Implementation

This chapter covers the implementation of yamir on both laptops and android smartphone.

We will first cover prerequisites for our implementation and then examine each of our components in turn final noting anyEc issues.

- kyamir – Linux kernel module for MANET IP router
- yamird – User-space daemon for MANET IP router
- Android app – GUI and backend
- SIP client - Linphone

4.1 Prerequisites

Building the project required

- Two Fedora 16 laptops, each with a Edimax EW-7612U USB Wi-Fi adapter
- Two HTC Desires, and one Samsung 2
- Kernel module headers for laptop builds, HTC Desire and Samsung S2
- Android SDK (for building JAVA apps)
- Android NDK (for building C/C++ code)

On our Linux build platform (Fedora 16, running 3.3 kernel) this means make, gcc and the kernel-devel rpms. For Android app development the android SDK is required and if you're using Eclipse for your IDE an ADT plugin as well [24].

Compiling code natively for an android handset requires an ARM tool-chain which includes things such as a compiler,linker, header files and libc. Although you can roll you're own most now use either the Android NDK or CodeSourcery cross compilers which allow people to build ARM binaries on their own workstation rather than the on the handsets [24-25].

Building kernel modules also requires the Linux kernel source headers to be installed and a Makefile entry that invokes the kernels kbuild system.

For Linux workstations the kernel headers are usually located in /lib/modules and the standard Kbuild mechanism can be used to compile the out-of-tree kernel module.

For mobile handsets you need a copy of kernel source code that matches that version used by the handset. These can be downloaded from manufacturer websites.

4.2 Building

Simply cd into the router folder and run make. For s2 and htc-desire devices the TARGET must be set and the resulting yamird and kymair.so copied to the android application res/raw folder.

- Linux workstation: `make`
- Samsung Galaxy S2: `make TARGET=samsungs2`
- HTC Desire: `make TARGET=htcdesire`

4.3 kyamir - Linux kernel module

kyamir is the Linux kernel module part of the MANET IP router. It uses netlink to communicate with user-space and the netfilter framework to intercept IP packets.

The module can accepts ifname parameter that it should monitor for IPv4 traffic.
e,g.

```
# insmod kyamir.ko ifname=wlan0
```

The network interface must be already up and have an IPv4 address or it will be ignored. As the module is mostly used on Android the default network interface is wlan0 .

kyamir code can be divided into three sections:

- Netlink: exchange netlink messages with user-space (yamird)
- Netfilter: intercept IP packet to trigger routing discovery
- Routing: how packets/routes are stored

Netlink

kyamir uses a netlink socket to exchange yamir messages with user-space.

- netlink_id : NETLINK_USERSOCK
- netlink_group: 1

The yamir message format have the following type:

```
struct yamir_msg { uint32_t addr, int ifindex};
```

mssage types sent by *kyamir* to *yamird*

- YAMIR_RT_NEED needs a route for IP packet (triggers route discovery)
- YAMIR_RT_INUSE reports a a route is being used (prevent expiry)
- YAMIR_RT_ERR: routing error has occurred (cant add route)

message types received by *kyamir* from *yamird*

- YAMR_RT_ADD route added for addr, Inject queued packets back into stack.
- YAMIR_RT_DEL route has been deleted. Discard queued packets for addr.
- YAMIR_RT_NONE no route for addr Discard queued packets, send ICMP unreachable

Netfilter

kyamir use the following netfilter hooks

- NF_INET_PRE_ROUTING
- NF_INET_LOCAL_OUT
- NF_INET_POST_ROUTING

For the NF_INET_PRE_ROUTING hook a route-need message is sent to yamird indicating the source ip address (saddr) is in use. If a route entry for the source address kyamir allows the IP packet to process (NF_ACCEPT) else it sends a route-err message to yamird and tell netfilter to drop the packet with a NF_DROP response code.
The NF_INET_LOCAL_OUT hook is used to initiate route discovery for locally created data packets. If a route exists for the packets destination IP address it is allowed to proceed (NF_ACCEPT) else the packet is queued and NF_STOLEN is reported back to kernel. If the queue for the IP address was previously empty a route-need message is sent to yamird.

The NF_INET_POST_ROUTING hook is used to send a route-inused message to yamird. This allows user-space to keep the active routes but quickly discard stale routes.

Routing

On startup a list_head queue is created for storing in IP packets waiting for route discovery. The queue entry simply store the socket buffer, destination IP address and i/f the packet came from. All read and write access to the queue is guarded by queue_lock which is a kernel spin-lock.

kyamir also maintains its own internal routing tables, as the Linux kernel 2.6 series used by the Android handsets did not provide a stable API for out-of-tree kernel modules to directly query the FIB. Using rtnetlink from within kyamir for route queries would have introduced additional overhead and unwanted latency in the netfilter packet filtering path.

4.4 yamird – user-space daemon

yamird is the user-space part of the MANET IP router that's responsible for both route discovery and route maintenance.

yamird requires the name of the network interface that the MANET is to be run on. Optionally it can be started in daemon mode and with logging (to stderr) turned on.

```
$ ./yamird -d -i wlan0 -l 10
```

yamird is a single threaded process that uses non-blocking sockets and poll() based event I/O to manage both Netlink and DYMO traffic via 3 sockets:

- kyamir netlink socket
- rtnetlink netlink socket
- dymo UDP socket

As yamird is is essentially I/O bound process a single-threaded architecture using poll() easily scales to handling large number of small packets. It also uses recvmmsg() to allow batch processing of packets to further reduce the overhead of kernel ABI calls.

The code uses a min-heap based timer API that provides millisecond granularity for DYMO protocol timeouts. The timer state used a fixed size and embedded state structure design that does not use malloc.

Finally *yamird* uses its own PacketBB (rfc5444) codec API to read/write DYMO messages.

yamird can can be divided up into 3 areas:

- kyamir: the netlink messages it sends/receives via kyamir
- rtnetlink: route updates it sends/receives via kernel rtnetlink
- dymo: dymo routing message its send/receives

KYAMIR

yamrid uses a user-mode netlink socket for kyamir communication.

```
fd = socket(AF_NETLINK, SOCK_RAW, NETLINK_USERSOCK);
```

Note only one kernel module that utilizes a user-mode socket can be loaded at any given time.

This was not an issue on our Android handsets and laptops as yamir and kyamir were the the only code using user-mode sockets for message exchange.

RTNETLINK

Access to the kernel routing tables is via an IPv4 route socket (rtnetlink) which uses the same netlink ABI as kyamir except the socket protocol is NETLINK_ROUTE.

```
fd = socket(AF_NETLINK, SOCK_RAW, NETLINK_ROUTE);
```

Updating a kernel route via rtnetlink is somewhat involved but it can be summarized as creating and sending a netlink message with its type set to one of the following values:

- RTM_NEWROUTE - create a kernel route
- RTM_DELROUTE - delete a kernel route

And setting some or all of the following field attributes:

- RTA_DST - destination address for which a route needs to be setup
- RTA_PRIORITY - the next hop count (or metric)
- RTA_OIF - next hop interface index
- RTA_GATEWAY - next hop IP address

DYMO

yamird implements the DYMO routing protocol and send and receives PacketBB encoded messages over UDP port 269.

Message are either multicast to the MANET all routers address 224.0.0.109 or unicast to a specific node IPv4 address. yamird uses multicast to relay and listen for route updates from its neighbours.

DYMO uses the following 3 MANET message types

- route-request (RREQ): msg-type 10
  if request is for know route route reply will be sent back
  if route is not known it will be relayed to neighbours

- route-reply (RREP) (msg-type 11)
  if reply has valid rouing info the local routing table will be update
  if reply is for remote node then it will be relayed
  if reply for local node route-need send route-add msg to kyamir

- router-error (RERR) msg-type 12
  all routes used by unreachable node will be discarded.
  The route message will be relayed if hop count not exceeded.

4.5 Android application

The Android application consists of 2 parts

- A backend with shell scripts and binaries for running the MANET
- A frontend GUI to control it

These are packaged together into an android apk file yamir-android.apk.
Note app needs a rooted handset (su installed) to start/stop the MANET.

Backend - shell scripts and binaries

This consists of

- yamir_sh script to start and stop MANET
- samsung_s2_sh and htc_desire_shd scripts to enable Ad-Hoc mode
- kyamir.ko compiled for both HTC Desire and Samsung S2
- yamird routing daemon compiled for ARM
- iwconfig to enable Ad-hoc mode (from wireless-tools)
- killall (from psmisc)

The app needs a kyamir kernel module and shell script for each Android device because kernel versions and Wi-Fi configurations vary.

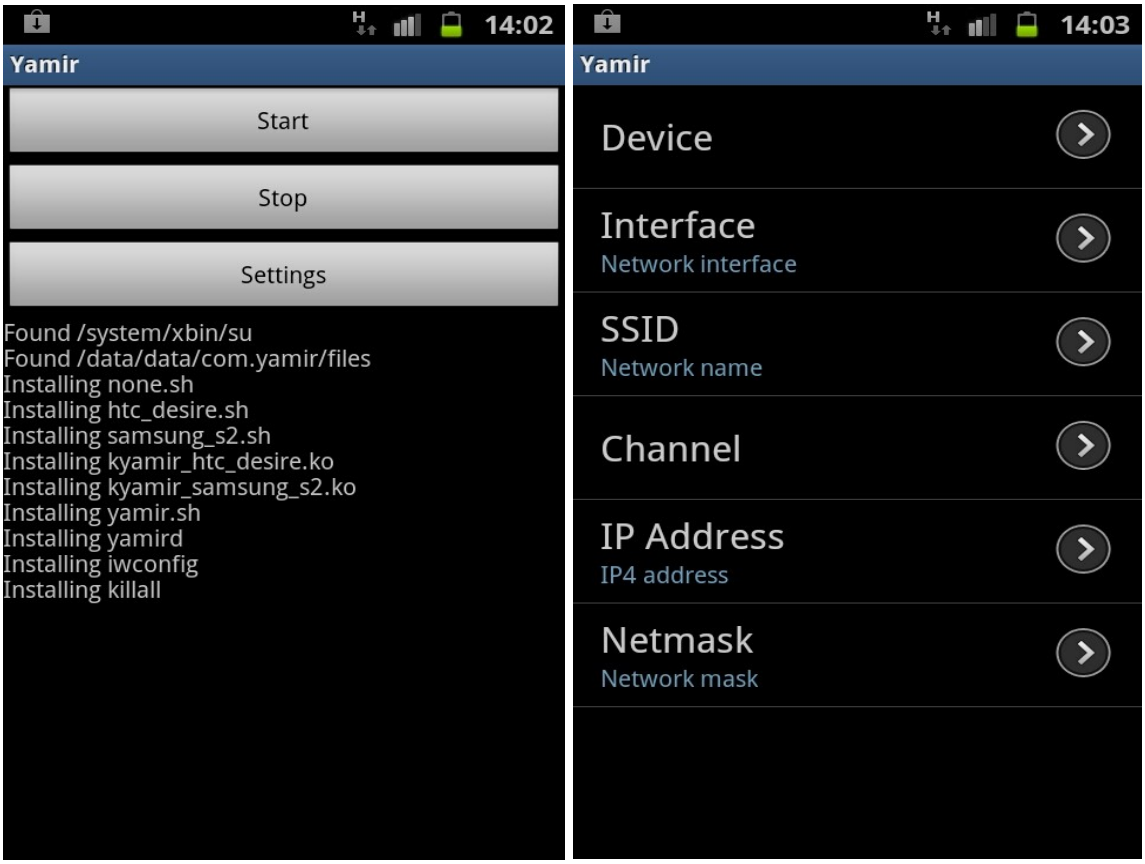
The iwconfig and killall are needed these are not standard on android handsets. As yamir, iwconfig, killall are all user-space binaries they can be safely compiled for generic ARM.

All of the binaries and shell scripts are located in the android apps raw resource folder where they must be first unpacked before use. This is because the Android apk build tool and the handset apk installer only allow native libraries, blacklisting executables.

The yamir.sh script used a config file created by the GUI and has start and stop options.

Start option	**Stop option**
loads the config file turns off the WiFi service load the wifi driver and its firmware (insmod) set the wlan.driver.status to “ok” brings down the interfaces (netcfg) enables 802.11 Ad-hoc mode (iwconfig) assigns the MANET ip address and netmask. brings the interface up loads the kyamir kernel module (via insmod) starts yamird in daemon mode (yamird -d) sets yamir.service to “started”	loads the config file shuts down the yamir process (killall) unloads the kymair kernel module (rmmod) unloads the wifi driver (disabling Ad-hoc) sets wlan.driver.status to “unloaded” sets yamir.service to “stopped”

Frontend - GUI



The GUI consists of 2 files YamirActivity and SettingsActivity.

YamirActvity contains most of the code for starting and stopping the MANET. On first being loaded by Android it locates its installation dir and the location of the su tool which is needed to launch the backend scripts with root access. It then unpack the backend binaries and shell scripts into the apps files work area with the correct permissions.

On pressing the “Start” button the code checks if the backend has been installed correctly and then writes the settings to a config file that is read by the yamir.sh script. It then executes with su the yamir.sh script with the start option and the config file path, displaying the results to the user in the status log.

Pressing “Stop” likewise calls yamir.sh with the stop option and the config file path.

SettingsActivity allows the following settings:

- **Device:** Used to select both the device script and kernel module to load.
- **Interface:** the network interface to use (default eth0)
- **SSID:** ad-hoc SSID network name to associate with (default yamir)
- **Channel**: The wireless channel to use (default 6)
- **IP Address:** The MANET IPv4 address to (default 192.168.1.10)
- **Netmask:** the address mask to use (default 255.255.255.0)

4.6 SIP Client

A patched version of Linphone is used for SIP calls on Android phones.

Steps to install Linphone on phone:

- Build the Linphone APK
- Enable side-loading on phone
- Install the Linphone APK on phone

Build the Linphone APK

The linphone folder contains makefile that will download Linphone, patch its source-code and build an APK for side-loading onto an android device.

```
$ cd linphone
$ make
```

Enable side-loading on phone

- Open Phone menu
- Goto Settings/Applications
- Check Unknown Sources
- Plug phone into USB port

Install the Linphone APK on phone

```
$ adb devices
$ adb install build/bin/LinphoneLauncherActivity-debug.apk
  Success
```

4.7 Issues

- Running yamird on Linux workstations
- Firewall blocking multicast DYMO traffic
- Building kyamir for Android handsets
- HTC-Desire firmware dropping Ad-hoc mode traffic
- Linphone adding VLAN tags on Samsung 2

Running yamird on Linux workstations

Running yamird as as non-root on Linux requires the following permissions:

- **cap_net_bind_service:** uses privileged port 269
- **cap_net_raw:** for SO_BINDTODEVICE
- **cap_net_admin:** for netlink multicast

To fix this use setcap as follows

```
$ sudo setcap cap_net_bind_service,cap_net_raw,cap_net_admin=+ep yamird
```

Firewall blocking multicast DYMO traffic

If Linux is using a firewall such as iptables it may have to be configured to allow IPv4 multicast traffic to pass through.

Initial tests between 2 Fedora Linux laptops failed due to multicast DMYO packets not being delivered to the yamird process even though wireshark showed the UDP packet arriving on the correct interface.

Subsequent investigation showed that iptables was the dropping traffic. A quick way to test if the firewall is dropping traffic is to run (iptables -F) which flushes the current ruleset. On Fedora 16 the following iptables command allows MANET multicast packets through:

```
$ sudo iptables -I INPUT 1 -p udp --dst "224.0.0.109" -j ACCEPT
```

Building kyamir for Android handsets

Too successfully build a kernel module for an Android handset requires compiler flags, kernel configuration (.config) and Makefile settings that are identical to those used by the android devices kernel or the module will simply fail to load.

For Samsung S2 the gingerbread kernel source provided for the gt-i9100 model (S2) contains a config file that had settings different to the one used for production handsets.

Although HTC Desire handsets have a readable /proc/config.gz the Froyo kernel source contains a Makefile with a EXTRAVERSION variable that needs to be set to the same value as used on the phone.

After fixing these problems the handset kernel then need to be cross compiled using an ARMtool-chain as mentioned in the prerequisites. Once the handset kernel headers have been generated kyamir needs to be cross compiled for each handset and copied into the android app res/raw folder.

HTC-Desire firmware dropping Ad-hoc mode traffic

Testing on HTC Desire handset we found that the DYMO packets where not sent out on the wireless interface. The problem was traced to the default firmware being loaded onto the bcm4329 chipset which always drops Ad-Hoc traffic.

The solution according to both android and xda-developer forums was to load the fw_bcm4329_ap.bin from a stock HTC Evo 4G device onto the chipset instead of the default fw_bcm4329.bin.

Linphone adding VLAN tags on Samsung S2

Initial tests of YAMIR using the Linphone SIP client running on a number of Linux laptops was successful with routes being setup and voice calls being received correctly.

However using the Linphone app on a Samsung S2 handset we found it would only intermittently use the connection.

We discovered the App was dependent on the handset's connectivity manager to report at least one available data network connection before it enabled SIP calls.

The data network connection always bound to 3G mobile instead of Wi-Fi. Switching off mobile network confirmed this. After modifying the Linphone backend (eXutils.c, misc.c) to alay use Wi-Fi we where able to make calls between the Samsung S2 and our laptops.

Using the modified version of Linphone we where also able to make calls between a pair of HTC desires and our laptops. However calls between the Samsung with HTC's failed. The initial SIP call setup traffic was being received by the Samsung but the reply's (confirmed by wireshark) where ignored by the HTC phone.

On closer examination it was found that Samsung SIP traffic contained 802.1Q or VLAN tags in the Ethernet frames which the HTC did not understand. The problem was located to Linphone submodules oRTP(rtpsession_inet.c) and exosip(eXtl_udp.c) always setting the IP_TOS option on the UDP socket they created for SIP and RTP traffic.

Disabling this removed the VLAN tag from Ethernet frames enabling SIP calls between the Samsung S2 and the HTC Desires.

Chapter 5

Conclusions and future work

The primary goal of building and deploying a functional MANET for both Linux laptops and android mobile devices was achieved.

Building a fully functional MANET router is a major task in itself but the amount of work required to configure Android handsets to use it was far greater than what was originally envisioned.

The added complexity of building and maintaining separate kernel modules for each handset combined with the problems encountered in switching on Ad-hoc mode and the need to modify a 3rd party VoIP application made the project goal almost unachievable.

The main problem is that Android handsets do not normally support 802.11 Ad-hoc mode and need to be *rooted* to enable ad-hoc support, run custom kernel modules and in some cases use different firmware than the factory defaults.

Even successfully configured handsets may fail if the users can update the operating system, as the kyamir kernel module would need to be rebuilt and the yamir app reinstalled.

Trying to support every kernel version released, even for a small number of Android devices, quickly becomes a maintenance nightmare. Additionally the reliance on a modified version of Linphone rather than the standard VoIP app make MANET support much more difficult.

Maintaining a MANET on Android handsets is truly only feasible for proprietary devices, where the Wi-Fi interface, kernel, and applications are controlled by the the supplier rather than the user.

802.11 Ad-hoc may be be soon displaced thanks to the recent Android 4.0 release which now contains direct support for Wi-Fi Direct although its a little unclear yet what form of ah-hoc networks are supported.

The adoption of 802.11s mesh mode may also simplify the support of MANET routers on future devices, provided the standard is fully supported by the handset's kernel.

Bibliography

[1] Yao-Nan Lien, Li-Cheng Chi, and Yuh-Sheng Shaw, “A Walkie-Talkie-Like Emergency Communication System for Catastrophic Natural Disasters”, 2009 10th International Symposium on Pervasive Systems, Algorithms, and Networks (ISPAN)

[2] The Dharamsala Community Wireless Mesh Network, http://drupal.airjaldi.com/node/56

[3] Maury Wright, “Wi-Fi Direct adds Peer-to-Peer Capabilities to Ubiquitous Wireless Network Technology Electronic Products”, http://www.digikey.com/us/en/techzone/wireless/resources/articles/Wi-Fi-Direct-adds-Peer-to-Peer-Capabilities.html

[4] Hongbo Zhou and Matt W. Mutka,Lionel M. Ni, “IP Address HandOff in the Manet”, INFOCOM 2004. Twenty-third AnnualJoint Conference of the IEEE Computer and Communications Societies

[5] Swarnapriyaa, U.G.; Vinodhini, V.; Anthoniraj, S.; Anand, R, “Auto Configuration in Mobile Ad Hoc Networks”, 2011 National Conference on Innovations in Emerging Technology (NCOIET)

[6] Sanket Nesargi, Ravi Prakash, “MANETconf: Configuration of Hosts in a MobileAd Hoc Network”, IEEE INFOCOM 2002,

[7] Sarkar, S. K. et al. ,"Ad Hoc Mobile Networks", Auerbach Publications, ISBN “978-1420062212”

[8] IETF Manet Working Group, http://datatracker.ietf.org/wg/manet/charter/

[9] “Optimized Link State Routing Protocol”, http://datatracker.ietf.org/doc/rfc3626

[10] M. M. Chandane, S. G. Bhirud and S.V. Bonde,“Analysis of AODV, DSR and DYMO Protocols for Wireless Sensor Network”,Journal of Computing,Volume 4, Issue 1, January 2012

[11] Ververidis, C.N.; Polyzos, G.C.,“Service Discovery for Mobile AD HOC Networks: A Survey of Issues and Techniques”, IEEE Communications Surveys & Tutorials, Volume 10, Issue 3 2008

[12] Patrick Stuedi, Marcel Bihr, Alain Remund, Gustavo Alonso, “SIPHoc: efficient SIP middleware for ad hoc networks”, MIDDLEWARE2007: Proceedings of the 8th ACM/IFIP/USENIX international conference on Middleware.

[13] Serhani, M.A.; Gadallah, Y. “A Service Discovery Protocol for Emergency Response Operations Using Mobile Ad Hoc Networks”, 2010 Sixth Advanced International Conference on Telecommunications (AICT).

[14] Alan B. Johnston, “SIP: Understanding the Session Initiation Protocol” , Artech House Telecommunications ,ISBN-13: “978-1607839958”

[15] Hiertz, G.R et al , “IEEE 802.11s: The WLAN Mesh Standard”, IEEE Wireless Communications 2010 Vol 17, Issue 1

[16] Serval Project, http://www.servalproject.org/

[17] Paul Gardner-Stephen, Swapna Palaniswamy,“Serval Mesh Software-WiFi Multi Model Management”,ACWR '11: Proceedings of the 1st International Conference on Wireless Technologies for Humanitarian Relief

[18] batman source - http://www.open-mesh.org/

[19] Leo Sicard,Matyas Markovics,Giannakis Manthios, “An Ad-hoc Network of Android Phones Using B.A.T.M.A.N.”, Project Report SPVC-E2010, IT-University Copenhagen, 2010

[20] Seither, D.; Konig, A.; Hollick, M, “Routing Performance of Wireless Mesh Networks: A Practical Evaluation of BATMAN Advanced” 2011 IEEE 36th Conference on Local Computer Networks (LCN).

[21] Linux packet filtering framework - www.netfilter.org.

[22] Dynamic MANET On-demand (DYMO) Routing
https://datatracker.ietf.org/doc/html/draft-ietf-manet-dymo-21

[23] “Generalized Mobile Ad Hoc Network (MANET) Packet/Message Format”, http://tools.ietf.org/html/rfc5444

[24] Android SDK/NDK, http://developer.android.com/sdk/index.html

[25] CodeSourcery, http://www.mentor.com/embedded-software/codesourcery

[26] “Compiling Kernel Modules”, http://tldp.org/LDP/lkmpg/2.6/html/x181.html

[27] W. Richard Stevens, “Unix Network Programming”, Volume 1 Second Edition ISBN-10: 013490012X, Chapter 20.2